

AWS Preparation — 10 Topic Theory Guide

Sep 23, 2026 · @Someone

How to use this guide

Study the 10 topics in order: each one builds on the previous, and together they cover the core of the AWS Solutions Architect – Associate syllabus. Every topic has the same shape: core concepts, how the pieces fit, comparison tables, common exam traps, and a self-check list.

#	Topic	Core question it answers	Suggested days
01	Account and billing	Who owns the resources and who pays?	2
02	IAM	Who can do what?	4
03	EC2 and networking	How do I run and scale servers?	5
04	VPC properly	How is traffic isolated and routed?	5
05	Storage and databases	Where does data live?	6
06	Serverless	How do I run code without servers?	4
07	Infrastructure as code	How do I build it repeatably?	3
08	Cost and observability	What does it cost and is it healthy?	3
09	Security	How is it protected end to end?	4
10	Certification	How do I pass the exam?	4

Total is about 40 study days at 1.5–2 hours per day. Pair every theory session with 20–30 minutes of hands-on work in a Free Tier account.

01 — Account and billing

An AWS account is the basic boundary for resources, security and billing; everything else in AWS lives inside one.

1.1 AWS global infrastructure

Unit	What it is	Why it matters
Region	A geographic area (e.g. ap-south-1 Mumbai, ap-southeast-1 Singapore)	Data residency, latency, service availability and price differ per Region
Availability Zone (AZ)	One or more isolated data centers inside a Region, linked by low-latency fiber	Deploy across 2+ AZs for high availability
Local Zone	AWS compute placed near a large city	Single-digit ms latency for local users
Edge location	CloudFront / Route 53 point of presence	Caching and DNS close to users

Choosing a Region follows four factors: compliance (where data must stay), latency to users, service availability, and price.

1.2 The account and the root user

- The root user is the email used to create the account. It has unrestricted access and cannot be limited by IAM policies.
- Use root only for a few tasks: changing account settings, closing the account, changing the support plan, restoring IAM admin access, some billing settings.
- Protect root: enable MFA, delete root access keys, never use it daily. Create an admin IAM user or, better, use IAM Identity Center.

1.3 Multi-account strategy with AWS Organizations

One account per workload or environment (dev, staging, prod) limits the blast radius of mistakes and makes cost tracking clean.

flowchart TD

```

M[Management account<br/>billing + org control] --> R[Root]
R --> S[OU: Security<br/>log archive, audit]
R --> W[OU: Workloads]
R --> SB[OU: Sandbox]
W --> D[Dev account]
W --> P[Prod account]

```

The management account sits at the top; Organizational Units (OUs) group accounts so policies can apply to many at once.

- **Consolidated billing:** one bill for all member accounts; usage is combined, so volume discounts and Reserved Instance / Savings Plan benefits are shared.
- **Service Control Policies (SCPs):** guardrails that set the *maximum* permissions in an account or OU. They never grant permission; they only limit. They do not affect the management account.
- **Tag policies** and **backup policies:** enforce standards across accounts.
- **AWS Control Tower:** sets up a best-practice multi-account "landing zone" automatically, with guardrails (preventive = SCPs, detective = AWS Config rules) and Account Factory for new accounts.

1.4 Pricing fundamentals

AWS charges on three main drivers: compute, storage, and data transfer out. Data transfer *into* AWS is generally free; data *out* to the internet and between Regions costs money; traffic between AZs usually costs a small amount too.

Pricing model	How it works	Best for
On-Demand	Pay per second/hour, no commitment	Short, unpredictable workloads
Savings Plans	Commit to \$/hour for 1 or 3 years	Steady usage across EC2, Fargate, Lambda
Reserved Instances	Commit to a specific instance type/Region for 1 or 3 years	Steady databases and servers
Spot	Spare capacity at up to ~90% off; can be interrupted with 2-minute notice	Fault-tolerant batch jobs, CI, big data

Pricing model	How it works	Best for
Dedicated Hosts	A physical server for you	Licensing (BYOL) and compliance

1.5 Support plans

Plan	Key features
Basic	Free; docs, forums, core Trusted Advisor checks
Developer	Business-hours email support for one contact
Business	24/7 phone/chat, full Trusted Advisor, <1 hour response for production down
Enterprise On-Ramp	Pool of Technical Account Managers, <30 min for business-critical
Enterprise	Designated TAM, <15 min for business-critical, concierge billing support

1.6 Billing and cost tools

- **Billing console / Bills:** monthly invoice by service and account.
- **Free Tier:** always-free services, 12-month trials, and short trials. Set a Free Tier usage alert.
- **AWS Budgets:** alerts when actual or forecasted cost or usage crosses a threshold (covered more in Topic 08).
- **Cost Explorer:** charts and forecasts of spend.
- **Cost allocation tags:** tags like `Project` or `Env` must be activated in Billing before they show in reports.
- **Pricing Calculator:** estimate cost before building.

1.7 Exam traps

- SCPs limit even the account's root user in member accounts, but never the management account.
- An SCP `Deny` wins over any IAM `Allow`.

- Consolidated billing gives one invoice and shared discounts; it does not share resources or permissions.
- "Lowest cost for interruptible jobs" means Spot; "steady 24/7 for 3 years" means Reserved/Savings Plans.

1.8 Self-check

- ☐ Explain Region vs AZ vs edge location
- ☐ List 4 tasks that need the root user
- ☐ Explain how SCPs combine with IAM policies
- ☐ Compare On-Demand, Savings Plans, RI and Spot
- ☐ Name the cheapest plan with 24/7 phone support (Business)

02 — IAM (Identity and Access Management)

IAM decides who (a principal) can perform which action on which resource, under which conditions; it is global and free.

2.1 Authentication vs authorization

- **Authentication** = proving who you are (password + MFA, access keys, federated login).
- **Authorization** = deciding what you may do, done by evaluating policies.

2.2 Identities

Identity	What it is	Credentials	Use it for
User	A person or app with long-term identity	Password, access keys	Legacy or break-glass; prefer Identity Center for people
Group	Collection of users	None (groups can't log in)	Attach policies once for many users
Role	Identity assumed temporarily	Short-lived STS credentials	EC2/Lambda access, cross-account, federation

A role has two policies: a **trust policy** (who may assume it) and a **permissions policy** (what it can do once assumed).

2.3 Policies

A policy is a JSON document with statements. Each statement has **Effect**, **Action**, **Resource**, optional **Condition**, and **Principal** for resource-based policies.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject"],
    "Resource": "arn:aws:s3:::reports-bucket/*",
    "Condition": {"Bool": {"aws:SecureTransport": "true"}}
  }]
}
```

Policy type	Attached to	Purpose
Identity-based (managed or inline)	User, group, role	Grant permissions
Resource-based	Resource (S3 bucket, SQS queue, KMS key, Lambda)	Grant access to other principals, incl. other accounts
Permission boundary	User or role	Maximum permissions an identity can ever get
SCP	Account / OU	Maximum permissions in the account
Session policy	An assumed-role session	Further narrow one session

AWS managed policies are maintained by AWS; customer managed policies are yours and reusable; inline policies live inside one identity and are deleted with it.

2.4 Policy evaluation logic

flowchart TD

```

A[Request] --> B{Explicit Deny<br/>anywhere?}
B -- Yes --> X[DENY]
B -- No --> C{Allowed by SCP,<br/>boundary, session?}
C -- No --> X
C -- Yes --> D{Allow in identity or<br/>resource policy?}
D -- Yes --> Y[ALLOW]
D -- No --> X

```

Three rules to memorize: everything starts as implicit deny; an explicit Allow overrides implicit deny; an explicit Deny beats everything. For cross-account access, both sides must allow: the caller's identity policy and the target's resource policy (or role trust).

2.5 STS and temporary credentials

- **AWS STS** issues short-lived credentials (access key, secret, session token), typically 15 minutes to 12 hours.
- `AssumeRole` : switch into a role, same or other account.
- `AssumeRoleWithWebIdentity` : exchange a web/OIDC token (Cognito, Google, GitHub Actions) for AWS credentials.
- `AssumeRoleWithSAML` : exchange a corporate SAML assertion.
- EC2 gets role credentials through an **instance profile**; apps read them from instance metadata (IMDSv2 recommended).

2.6 Workforce and customer identity

- **IAM Identity Center** (former AWS SSO): one login for employees across all Organization accounts, with permission sets; connects to Active Directory, Okta, Entra ID.
- **Amazon Cognito**: sign-up/sign-in for your app's end users. *User pools* = user directory and tokens (JWT); *identity pools* = exchange tokens for temporary AWS credentials.

2.7 Security tools inside IAM

- **IAM Access Analyzer**: finds resources shared outside your account and validates policies; can generate least-privilege policies from CloudTrail activity.
- **Credential report**: account-wide list of users and credential status.

- **Access Advisor / last accessed:** shows unused permissions to remove.
- **Password policy** and **MFA** (virtual app, hardware key, passkey).

2.8 Best practices

1. Lock away the root user; enable MFA.
2. Humans use Identity Center or federation, not long-term IAM users.
3. Workloads use roles, never embedded access keys.
4. Grant least privilege; start with managed policies, then tighten.
5. Use conditions (source IP, MFA present, tags, `aws:PrincipalOrgID`).
6. Rotate or remove unused credentials; review with Access Analyzer.

2.9 Exam traps

- "Application on EC2 needs S3 access" → IAM role via instance profile, never keys in code.
- "Give another account access" → cross-account role or resource-based policy.
- A permission boundary does not grant anything; effective permission = identity policy + boundary.
- IAM is global; you don't pick a Region for users or roles.
- Groups cannot be nested and cannot be a `Principal`.

2.10 Self-check

- ☐ Draw the policy evaluation flow from memory
- ☐ Explain trust policy vs permissions policy
- ☐ Write a policy allowing read-only access to one S3 prefix
- ☐ Explain Cognito user pool vs identity pool
- ☐ Explain when to use a permission boundary

03 — EC2 and networking

EC2 gives you virtual servers; load balancers, Auto Scaling, Route 53 and CloudFront turn single servers into a scalable, highly available application.

3.1 EC2 building blocks

- **AMI (Amazon Machine Image):** template with OS + software. Regional; copy to use in another Region. You can build your own "golden AMI".
- **Instance type:** CPU, memory, network size, named like `m7g.large` = family `m`, generation `7`, attribute `g` (Graviton/ARM), size `large`.
- **Key pair:** SSH login; or use **Session Manager** (no open port 22 needed).
- **User data:** script that runs at first boot (install packages, pull code).
- **Instance metadata (IMDS):** `169.254.169.254` gives instance info and role credentials; enforce IMDSv2.

Family	Letter	Optimized for	Example use
General purpose	t, m	Balanced	Web servers, small apps (t = burstable credits)
Compute optimized	c	CPU	Batch, gaming, high-traffic APIs
Memory optimized	r, x, z	RAM	In-memory caches, large databases
Storage optimized	i, d	Local disk IOPS	NoSQL, data warehousing
Accelerated	p, g, inf, trn	GPU / ML chips	ML training and inference

3.2 Instance storage

Type	Persistence	Key facts
EBS volume	Persists independent of the instance	Network block storage, bound to one AZ; snapshots go to S3 and can be copied across Regions
Instance store	Lost on stop/terminate	Physically attached, very fast; for temp/cache data
EFS	Persistent, shared	NFS file system mounted by many instances across AZs (Topic 05)

EBS types: **gp3** (general SSD, default, IOPS set independently of size), **io2 Block Express** (highest IOPS, critical DBs; supports Multi-Attach), **st1** (throughput HDD, big sequential data), **sc1** (cold HDD, cheapest). EBS encryption uses KMS and covers data at rest, snapshots and data in transit to the instance.

3.3 Instance lifecycle and purchasing

- States: pending → running → stopping → stopped → terminated. **Hibernate** saves RAM to EBS for fast resume.
- Billing stops for compute when stopped; EBS storage is still billed.
- Purchase options: On-Demand, Savings Plans, Reserved, Spot, Dedicated Instances/Hosts, Capacity Reservations (see Topic 01).
- **Spot Fleet / EC2 Fleet**: mix Spot and On-Demand across many instance types for resilience.

3.4 Placement groups

Strategy	Layout	Use for
Cluster	Close together in one AZ	Lowest latency, HPC
Spread	Each instance on distinct hardware (max 7 per AZ)	Small number of critical instances
Partition	Groups on separate racks	Large distributed systems: Kafka, HDFS, Cassandra

3.5 Elastic Load Balancing

Load balancer	OSI layer	Routes by	Pick it when
Application (ALB)	7 (HTTP/HTTPS, gRPC, WebSocket)	Path, host, header, query	Web apps, microservices, containers
Network (NLB)	4 (TCP/UDP/TLS)	IP and port	Extreme performance, static IP, millions of req/s
Gateway (GWLB)	3 (IP, GENEVE)	Flows to appliances	Firewalls and inspection appliances

Key concepts: **listeners** (port + protocol), **target groups** (instances, IPs, Lambda), **health checks**, **cross-zone load balancing**, **sticky sessions**, **connection draining (deregistration delay)**, and **TLS termination** with certificates from ACM.

3.6 Auto Scaling

An Auto Scaling group (ASG) keeps a desired number of instances between min and max, replacing unhealthy ones automatically, built from a **launch template**.

```

flowchart LR
    U[Users] --> ALB[ALB]
    ALB --> A1[EC2 in AZ-a]
    ALB --> A2[EC2 in AZ-b]
    CW[CloudWatch alarm] --> ASG[Auto Scaling group]
    ASG --> A1
    ASG --> A2
  
```

The ALB spreads traffic; the ASG adds or removes instances in both AZs when CloudWatch metrics change.

- **Target tracking:** keep a metric at a value (e.g. CPU at 50%) — the default choice.
- **Step / simple scaling:** act on alarm thresholds.
- **Scheduled:** known patterns (office hours).
- **Predictive:** ML forecast of recurring load.
- Cooldown / warm-up periods prevent flapping; **warm pools** keep pre-initialized instances ready.

3.7 DNS with Route 53

- Records: A (IPv4), AAAA (IPv6), CNAME (name to name, not at zone apex), **Alias** (AWS-specific, works at apex, free queries to AWS resources like ALB, CloudFront, S3 website).
- Hosted zones: public (internet) and private (inside VPCs).

Routing policy	Behavior
Simple	One record, no health checks
Weighted	Split traffic by percentage (canary, A/B)
Latency	Send user to the lowest-latency Region
Failover	Primary/secondary with health checks (active-passive DR)
Geolocation	Route by user's country/continent
Geoproximity	Route by distance with a bias
Multivalue answer	Up to 8 healthy records, client-side balancing
IP-based	Route by client CIDR

3.8 Edge networking

- **CloudFront**: CDN caching at edge locations; origins can be S3 (secured with Origin Access Control), ALB, EC2 or any HTTP server. Supports HTTPS, signed URLs/cookies, geo-restriction, WAF integration.
- **Global Accelerator**: two static anycast IPs routing over the AWS backbone to the best Regional endpoint; good for non-HTTP (TCP/UDP) and fast failover, no caching.

3.9 Exam traps

- "Keep data after instance termination" → EBS (with DeleteOnTermination off) or S3, not instance store.
- "EBS in another AZ" → snapshot, then create a volume in that AZ.
- "Path-based routing /api and /images" → ALB. "Static IP / UDP / ultra-low latency" → NLB.
- "Point the zone apex (example.com) at an ALB" → Alias record, not CNAME.

- "Static IPs for a global app, fast Regional failover" → Global Accelerator.

3.10 Self-check

- ☐ Decode an instance name like `r6i.2xlarge`
- ☐ Compare gp3, io2, st1, sc1
- ☐ Choose ALB vs NLB vs GWLB for 3 scenarios
- ☐ Explain the 4 Auto Scaling policy types
- ☐ List all 8 Route 53 routing policies

04 — VPC properly

A VPC is your private, isolated network in one Region: you choose the IP range, split it into subnets per AZ, and control every route in and out.

4.1 IP addressing and CIDR

- CIDR notation `10.0.0.0/16` = first 16 bits fixed → $2^{(32-16)} = 65,536$ addresses. `/24` = 256 addresses.
- VPC size: between `/16` and `/28`. Use private ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.
- Plan non-overlapping CIDRs across VPCs and on-premises from day one; overlapping ranges cannot be peered.
- AWS reserves **5 addresses** in every subnet: network, VPC router, DNS, future use, broadcast. A `/24` subnet gives 251 usable IPs.
- IPv6: AWS assigns a `/56` to the VPC, `/64` per subnet; all IPv6 addresses are public (control with egress-only IGW).

4.2 Subnets and route tables

A subnet lives in exactly one AZ. It is "public" only because its route table sends `0.0.0.0/0` to an Internet Gateway; otherwise it is private.

flowchart TD

```

I[Internet] <--> IGW[Internet Gateway]
IGW <--> PUB[Public subnet<br/>ALB, NAT GW]
PUB --> APP[Private app subnet<br/>EC2, ECS]
APP --> DB[Private data subnet<br/>RDS]
APP -- outbound only --> NAT[NAT Gateway]
NAT --> IGW

```

The classic three-tier layout: repeat each subnet tier in at least two AZs for high availability.

Route table	Destination	Target
Public	10.0.0.0/16	local
Public	0.0.0.0/0	igw-xxxx
Private	10.0.0.0/16	local
Private	0.0.0.0/0	nat-xxxx

The `local` route is automatic and cannot be removed. The most specific route (longest prefix) wins.

4.3 Getting to and from the internet

- **Internet Gateway (IGW):** one per VPC, horizontally scaled, allows two-way internet traffic for resources with public IPs.
- **Public IP / Elastic IP:** a public IP changes on stop/start; an Elastic IP is static (charged; all public IPv4 addresses are billed hourly).
- **NAT Gateway:** lets private subnets reach the internet outbound only. Managed, lives in a public subnet in one AZ — deploy one per AZ for resilience. Charged per hour + per GB.
- **NAT instance:** older self-managed EC2 alternative; you must disable source/destination check.
- **Egress-only Internet Gateway:** the IPv6 equivalent of NAT.

4.4 Security Groups vs Network ACLs

Feature	Security Group	Network ACL
Level	Instance / ENI	Subnet
State	Stateful (return traffic auto-allowed)	Stateless (allow both directions explicitly)
Rules	Allow only	Allow and Deny
Evaluation	All rules evaluated together	Numbered order, first match wins
Default	Deny all inbound, allow all outbound	Default NACL allows all
Can reference	Other security groups	CIDR ranges only

With NACLs remember **ephemeral ports** (1024–65535) for return traffic. To block a single bad IP, use a NACL Deny — security groups cannot deny.

4.5 Private access to AWS services: VPC endpoints

Endpoint type	Services	How	Cost
Gateway endpoint	S3 and DynamoDB only	Entry in route table	Free
Interface endpoint (PrivateLink)	Most AWS services and your own/partner services	ENI with private IP in your subnet	Hourly + per GB

Endpoints keep traffic off the internet and can remove the need for NAT (and its cost). Endpoint policies restrict which resources can be reached.

4.6 Connecting VPCs

- **VPC peering:** private one-to-one link, same or different account/Region. Not transitive ($A \leftrightarrow B$ and $B \leftrightarrow C$ does not give $A \leftrightarrow C$). No overlapping CIDRs. Update route tables on both sides.
- **Transit Gateway:** hub-and-spoke router for hundreds of VPCs and on-premises links; transitive; route tables inside TGW allow segmentation; supports inter-Region peering.

- **PrivateLink**: expose one service (behind an NLB) to consumer VPCs without full network access; works with overlapping CIDRs.

4.7 Hybrid connectivity

Option	Path	Setup time	Bandwidth	Use for
Site-to-Site VPN	Encrypted IPsec over internet	Minutes-hours	Up to ~1.25 Gbps per tunnel	Quick, cheap, backup link
Direct Connect (DX)	Private dedicated line	Weeks	1, 10, 100 Gbps dedicated (hosted: smaller)	Consistent latency, large data
DX + VPN	VPN over DX	Weeks	DX speed	Private line plus encryption
Client VPN	OpenVPN from laptops	Minutes	Per user	Remote workers

Direct Connect is not encrypted by default. Use a **Direct Connect Gateway** to reach VPCs in many Regions, and a VPN as a backup path for resilience. VPN connects to a **Virtual Private Gateway** (one VPC) or Transit Gateway (many).

4.8 Visibility and DNS

- **VPC Flow Logs**: capture IP traffic metadata (source, destination, port, ACCEPT/REJECT) at VPC, subnet or ENI level, to CloudWatch Logs or S3. They do not capture packet contents.
- **Traffic Mirroring**: copy actual packets for deep inspection.
- **Reachability Analyzer**: tests whether a path between two resources works and why not.
- **Route 53 Resolver**: inbound/outbound endpoints for DNS between VPC and on-premises.
- **Network Firewall**: managed stateful firewall with IDS/IPS rules for the VPC.

4.9 Exam traps

- "Private instances need to download patches" → NAT Gateway in a public subnet + route from private subnet.

- "Private access to S3 with no NAT cost" → Gateway endpoint.
- "Connect 50 VPCs and on-premises" → Transit Gateway, not peering mesh.
- "Traffic reaches instance but reply is blocked" → NACL outbound missing ephemeral ports.
- "Highly available NAT" → one NAT Gateway per AZ.

4.10 Self-check

- ☐ Calculate usable IPs in a /26 subnet ($64 - 5 = 59$)
- ☐ Build a 2-AZ, 3-tier VPC on paper with route tables
- ☐ Explain stateful vs stateless with an example
- ☐ Choose peering vs Transit Gateway vs PrivateLink
- ☐ Compare VPN and Direct Connect

05 — Storage and databases

Pick storage by access pattern: object (S3) for files and data lakes, block (EBS) for one server's disk, file (EFS/FSx) for shared folders, and a database matched to the query shape.

5.1 Amazon S3 fundamentals

- Objects (up to 5 TB each) stored in **buckets**; bucket names are globally unique, but a bucket lives in one Region.
- Key = full path name (`reports/2026/jan.csv`); S3 is flat, "folders" are just prefixes.
- Durability 99.999999999% (11 nines); strong read-after-write consistency.
- Upload over 100 MB → use **multipart upload**; long-distance uploads → **Transfer Acceleration**.

Storage class	Access pattern	Retrieval	Notes
Standard	Frequent	Instant	Default
Intelligent-Tiering	Unknown / changing	Instant (archive tiers optional)	Auto-moves objects; small monitoring fee
Standard-IA	Infrequent, multi-AZ	Instant, retrieval fee	30-day minimum

Storage class	Access pattern	Retrieval	Notes
One Zone-IA	Infrequent, re-creatable	Instant, retrieval fee	One AZ only
Glacier Instant Retrieval	Quarterly access	Milliseconds	90-day minimum
Glacier Flexible Retrieval	Archive	Minutes to 12 hours	90-day minimum
Glacier Deep Archive	Long-term archive	12–48 hours	Cheapest, 180-day minimum
Express One Zone	Very high performance	Single-digit ms	Directory buckets, one AZ

Lifecycle rules move objects between classes or expire them after N days (e.g. Standard → IA at 30 days → Glacier at 90 → delete at 365).

5.2 S3 security and data protection

- **Block Public Access:** on by default; keep it on unless you truly host public files.
- Access control: IAM policies + **bucket policies** (resource-based). ACLs are legacy; "Bucket owner enforced" disables them.
- Encryption at rest: SSE-S3 (default), SSE-KMS (audit + key control), DSSE-KMS, SSE-C (you supply key), or client-side.
- **Versioning:** keeps every version; protects from overwrite/delete. **MFA Delete** adds protection.
- **Object Lock (WORM):** governance or compliance mode for regulated retention.
- **Replication:** Cross-Region (CRR) or Same-Region (SRR); needs versioning on both buckets.
- **Pre-signed URLs:** temporary access to one object without making it public.
- Event notifications → Lambda, SQS, SNS, EventBridge.

5.3 File and hybrid storage

Service	Protocol	Use for
EFS	NFS (Linux)	Shared files across many EC2/containers, multi-AZ, auto-scales
FSx for Windows File Server	SMB	Windows apps, Active Directory integration
FSx for Lustre	Lustre	HPC, ML training, links to S3
FSx for NetApp ONTAP	NFS, SMB, iSCSI	Lift-and-shift NetApp workloads
FSx for OpenZFS	NFS	Migrating ZFS workloads
Storage Gateway	NFS/SMB/iSCSI/VTL	On-premises apps using S3 as backend
DataSync	—	Online data transfer/migration to S3, EFS, FSx
Snow Family	Physical device	Offline transfer of TBs-PBs
AWS Backup	—	Central backup policies across services and accounts

5.4 Relational databases: RDS and Aurora

- **RDS engines:** MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2. AWS manages patching, backups, failover; you cannot SSH into the host (RDS Custom is the exception).
- **Multi-AZ:** synchronous standby in another AZ for **high availability**; automatic failover (~1-2 minutes). The standby does not serve reads (in the classic single-standby deployment).
- **Read replicas:** asynchronous copies for **read scaling**; can be cross-Region; can be promoted.
- **Backups:** automated (point-in-time restore, retention up to 35 days) and manual snapshots (kept until deleted).
- **RDS Proxy:** pools connections; great with Lambda; faster failover.
- **Aurora:** AWS-built MySQL/PostgreSQL-compatible engine; storage auto-grows to 128 TiB, 6 copies across 3 AZs, up to 15 low-lag replicas, **Aurora Serverless v2** (auto-scaling capacity) and **Global Database** (cross-Region, ~1 second lag, fast DR).

Need	Choose
Survive an AZ failure	Multi-AZ
Scale read traffic	Read replicas
Disaster recovery in another Region	Cross-Region replica or Aurora Global Database
Spiky or unpredictable DB load	Aurora Serverless v2

5.5 NoSQL and purpose-built databases

DynamoDB is a serverless key-value/document database with single-digit ms latency at any scale.

- Table → items → attributes. **Primary key** = partition key, or partition + sort key. Design for access patterns first.
- Indexes: **GSI** (different partition key, create any time) and **LSI** (same partition key, create at table creation).
- Capacity: **on-demand** (pay per request) or **provisioned** (RCU/WCU with auto scaling).
- Features: **DAX** (in-memory cache, microseconds), **Streams** (change feed to Lambda), **Global Tables** (multi-Region active-active), TTL, PITR backups, transactions.

Service	Type	Typical use
ElastiCache (Redis OSS / Valkey / Memcached)	In-memory cache	Session store, caching DB queries, leaderboards
MemoryDB	Durable in-memory	Redis-compatible primary database
DocumentDB	Document (MongoDB-compatible)	JSON documents
Neptune	Graph	Social networks, fraud detection
Keyspaces	Wide-column (Cassandra)	Cassandra workloads
Timestream	Time series	IoT, metrics

Service	Type	Typical use
Redshift	Data warehouse (columnar)	Analytics, BI, OLAP
OpenSearch	Search	Full-text search, log analytics

5.6 Choosing a database

flowchart TD

```

Q{Data shape?} --> R[Relational, joins,<br/>transactions]
Q --> K[Key-value at<br/>massive scale]
Q --> A[Analytics over<br/>huge data]
R --> RDS[RDS / Aurora]
K --> DDB[DynamoDB]
A --> RS[Redshift / Athena]
```

Then add a cache (ElastiCache or DAX) for hot reads, and pick specialized engines only for graph, time series or search needs.

5.7 Exam traps

- "High availability" for RDS → Multi-AZ; "improve read performance" → read replicas.
- "Shared file system for Linux instances across AZs" → EFS, not EBS.
- "Rarely accessed, must be retrieved in milliseconds" → Glacier Instant Retrieval.
- "Serverless, millisecond, key-value at any scale" → DynamoDB.
- "Query S3 data with SQL without loading" → Athena.
- "Unknown access pattern" → S3 Intelligent-Tiering.

5.8 Self-check

- ☐ Rank S3 storage classes by cost and retrieval time
- ☐ Design a lifecycle policy for log files
- ☐ Explain Multi-AZ vs read replica
- ☐ Design a DynamoDB key for "orders by customer by date"
- ☐ Match 6 workloads to the right database

06 — Serverless

Serverless means no servers to manage, automatic scaling (including to zero), built-in availability, and paying only for actual use; AWS Lambda is the center, joined by managed messaging and API services.

6.1 AWS Lambda

- Runs a function in response to an event; runtimes include Node.js, Python, Java, .NET, Go, Ruby, or a container image (up to 10 GB).
- Limits to memorize: max **15 minutes** timeout, memory **128 MB–10 GB** (CPU scales with memory), `/tmp` up to 10 GB, deployment package 50 MB zipped / 250 MB unzipped.
- Billing: number of requests + GB-seconds of duration.
- **Execution role**: the IAM role that gives the function its permissions.
- **Cold start**: first request initializes a new environment; reduce with smaller packages, **Provisioned Concurrency**, or **SnapStart** (Java, Python, .NET).
- **Concurrency**: default account limit per Region (commonly 1,000, raisable); **reserved concurrency** guarantees and caps one function.
- **Layers**: share libraries across functions. **Versions and aliases**: immutable versions, aliases for traffic shifting (canary deploys).
- VPC access: attach the function to private subnets to reach RDS; it then needs NAT or endpoints for internet/AWS APIs.

Invocation type	Example sources	Error behavior
Synchronous	API Gateway, ALB, SDK call	Caller receives the error and retries
Asynchronous	S3 events, SNS, EventBridge	Lambda retries twice, then to a DLQ or on-failure destination
Poll-based (event source mapping)	SQS, Kinesis, DynamoDB Streams, Kafka	Lambda polls in batches; retry per source rules

6.2 API Gateway

Type	Best for	Notes
HTTP API	Simple, low-cost REST/proxy to Lambda	Cheaper, faster, JWT authorizers
REST API	Full features	Usage plans + API keys, request validation, caching, WAF, private APIs
WebSocket API	Real-time two-way apps	Chat, live dashboards

Features: stages (dev/prod), throttling (default ~10,000 req/s per account per Region), authorizers (IAM, Cognito, Lambda), CORS, custom domains, and integration timeout of 29 seconds by default.

6.3 Messaging and integration: decoupling

Service	Model	Key idea	Use when
SQS Standard	Queue (pull)	At-least-once, best-effort order, nearly unlimited throughput	Buffer work between producers and consumers
SQS FIFO	Queue (pull)	Exactly-once processing, strict order per message group	Orders, payments
SNS	Pub/sub (push)	One message fans out to many subscribers	Notifications, fan-out
EventBridge	Event bus	Rules route events by content; SaaS and AWS events; scheduler	Event-driven apps across services
Kinesis Data Streams	Stream	Ordered, replayable, real-time, shards	Clickstreams, IoT, analytics
Data Firehose	Delivery stream	Load streaming data into S3, Redshift, OpenSearch	Near-real-time ingestion
Amazon MQ	Broker	ActiveMQ/RabbitMQ protocols	Migrating existing broker apps

SQS concepts: **visibility timeout** (message hidden while being processed), **long polling** (reduce empty responses), **dead-letter queue** (after N failed receives), message retention 1 minute–14 days, max message 256 KB.

Fan-out pattern: SNS topic → several SQS queues, so each consumer gets its own durable copy.

6.4 Orchestration with Step Functions

Step Functions coordinates multi-step workflows as a state machine, with retries, error catching, parallel branches, waits and human approval.

```
stateDiagram-v2
    [*] --> ValidateOrder
    ValidateOrder --> ChargePayment
    ChargePayment --> ShipOrder: success
    ChargePayment --> NotifyFailure: error
    ShipOrder --> [*]
    NotifyFailure --> [*]
```

Standard workflows run up to 1 year, exactly-once; **Express** workflows are high-volume, up to 5 minutes, cheaper. Orchestration (Step Functions) suits ordered business processes; choreography (EventBridge) suits loosely coupled services reacting to events.

6.5 Serverless containers

- **ECS** (AWS container orchestrator) and **EKS** (managed Kubernetes) can run on EC2 or on **Fargate**.
- **Fargate**: serverless compute for containers — no servers to patch; pay per vCPU and memory per second.
- **App Runner**: deploy a web app from source or image with almost no configuration.
- Choose Lambda for short event-driven tasks; choose Fargate for long-running services or jobs longer than 15 minutes.

6.6 A typical serverless web architecture

```

flowchart LR
    U[Browser] --> CF[CloudFront + S3<br/>static site]
    U --> APIGW[API Gateway]
    APIGW --> COG[Cognito<br/>auth]
    APIGW --> L[Lambda]
    L --> DDB[DynamoDB]
    L --> EB[EventBridge]
    EB --> SQS[SQS] --> W[Worker Lambda]
  
```

The static front end is served from S3 via CloudFront; the API is Lambda behind API Gateway; slow work goes through events and queues.

6.7 Exam traps

- "Job runs longer than 15 minutes" → not Lambda; use Fargate, Batch, or Step Functions to split it.
- "Decouple and absorb traffic spikes" → SQS. "Send one message to many" → SNS (fan-out).
- "Strict ordering, no duplicates" → SQS FIFO.
- "Replay data, multiple real-time consumers" → Kinesis Data Streams.
- "Too many DB connections from Lambda" → RDS Proxy.
- "Coordinate steps with retries and branches" → Step Functions.

6.8 Self-check

- ☐ Recite Lambda's key limits
- ☐ Explain sync vs async vs poll-based invocation
- ☐ Compare SQS, SNS, EventBridge, Kinesis
- ☐ Explain the visibility timeout and DLQ
- ☐ Draw a serverless web app from memory

07 — Infrastructure as code (IaC)

IaC defines infrastructure in version-controlled files so every environment is built the same way, reviewed like code, and recreated in minutes.

7.1 Why IaC

- **Repeatability:** dev, staging and prod built from the same template.
- **Version control:** Git history, pull-request reviews, rollback.
- **Automation:** CI/CD pipelines deploy infrastructure changes.
- **Documentation:** the code is the current truth of the system.
- **Disaster recovery:** rebuild a whole environment in another Region.

Two styles: **declarative** (describe the end state; tool works out the steps — CloudFormation, Terraform) and **imperative** (write the steps — scripts, SDK calls). Declarative is the norm.

7.2 AWS CloudFormation

CloudFormation reads a YAML/JSON **template** and creates a **stack**: a group of resources managed as one unit, with automatic rollback on failure.

```
AWSTemplateFormatVersion: '2010-09-09'
Parameters:
  EnvName:
    Type: String
    AllowedValues: [dev, prod]
Resources:
  AppBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub 'myapp-${EnvName}-assets'
      VersioningConfiguration:
        Status: Enabled
Outputs:
  BucketArn:
    Value: !GetAtt AppBucket.Arn
  Export:
    Name: !Sub '${EnvName}-bucket-arn'
```

Template section	Purpose
Parameters	Inputs at deploy time
Mappings	Static lookup tables (e.g. AMI per Region)
Conditions	Create resources only in some cases (prod only)

Template section	Purpose
Resources	The only required section
Outputs	Values to show or export to other stacks

Intrinsic functions: `!Ref`, `!GetAtt`, `!Sub`, `!Join`, `!If`, `!FindInMap`, `!ImportValue`.

Key features:

- **Change sets:** preview what an update will add, modify or replace before executing.
- **Drift detection:** finds resources changed manually outside CloudFormation.
- **Nested stacks:** reuse templates as components. **Cross-stack references:** export/import outputs.
- **StackSets:** deploy one stack to many accounts and Regions (works with Organizations).
- **DeletionPolicy** (`Retain`, `Snapshot`) and **UpdateReplacePolicy:** protect data such as databases.
- **Stack policies** and **termination protection:** prevent accidental changes.
- Helper scripts (`cfn-init`, `cfn-signal`) and **CreationPolicy** for EC2 bootstrapping.

7.3 Higher-level tools

Tool	Language	Output	Best for
CloudFormation	YAML/JSON	Stacks	Native, no extra tooling
AWS SAM	YAML (CloudFormation extension)	CloudFormation	Serverless apps: Lambda, API, DynamoDB; local testing
AWS CDK	TypeScript, Python, Java, C#, Go	Synthesizes CloudFormation	Developers wanting loops, classes and reusable constructs
Terraform	HCL	Its own state file	Multi-cloud and many SaaS providers
Pulumi	General languages	Its own state	Multi-cloud with real code

CDK concepts: **App → Stacks → Constructs**. L1 constructs = raw CloudFormation resources; L2 = sensible defaults; L3 = patterns (e.g. an ALB + Fargate service in one line). `cdk synth` makes the template, `cdk diff` previews, `cdk deploy` applies.

Terraform concepts: **providers, resources, modules, state** (store remotely in S3 with locking), `plan` then `apply`.

7.4 Deployment pipeline

```

flowchart LR
    G[Git push] --> B[Build + lint<br/>cfn-lint / tflint]
    B --> T[Security scan<br/>checkov / cfn-guard]
    T --> P[Plan / change set]
    P --> A[Manual approval]
    A --> D[Deploy to prod]
  
```

Every infrastructure change flows through review and automated checks before it touches production. AWS tools for this: CodePipeline, CodeBuild, CodeDeploy (or GitHub Actions with OIDC role assumption).

Deployment strategies: **all-at-once**, **rolling**, **blue/green** (two environments, switch traffic), **canary** (small % first), **immutable** (new instances, then swap).

7.5 Related configuration services

- **Systems Manager**: Parameter Store (config values), Run Command, Patch Manager, Session Manager, Automation runbooks.
- **Elastic Beanstalk**: PaaS — upload code, AWS builds EC2, ASG, ALB for you (uses CloudFormation underneath).
- **Service Catalog**: approved templates that teams can launch self-service.

7.6 Best practices

1. Never change IaC-managed resources in the console; fix drift through code.
2. Parameterize environments; no hard-coded account IDs or Regions.
3. Keep secrets out of templates; reference Secrets Manager or Parameter Store.
4. Split stacks by lifecycle (network, data, application).
5. Protect stateful resources with `DeletionPolicy: Retain` / snapshots.
6. Always preview (change set / plan) before applying to prod.

7.7 Exam traps

- "Deploy the same stack to many accounts and Regions" → StackSets.
- "Preview changes before updating a stack" → change sets.
- "Someone changed a security group manually" → drift detection.
- "Keep the database when the stack is deleted" → DeletionPolicy Retain or Snapshot.
- "Developers want to write infrastructure in TypeScript" → CDK.
- "Simplest way to deploy a web app without managing infrastructure details" → Elastic Beanstalk.

7.8 Self-check

- ☐ Write a template with a parameter, a resource, and an output
- ☐ Explain change sets vs drift detection
- ☐ Compare CloudFormation, SAM, CDK, Terraform
- ☐ Explain blue/green vs canary
- ☐ Explain why Terraform state must be stored remotely with locking

08 — Cost and observability

You cannot optimize what you cannot see: observability tells you whether the system is healthy, and cost tooling tells you what that health costs.

8.1 Cost optimization principles

From the Well-Architected Cost Optimization pillar:

1. **Practice cloud financial management (FinOps):** someone owns cost; engineers see their spend.
2. **Adopt a consumption model:** pay only for what you use; stop dev resources at night.
3. **Measure efficiency:** cost per order, per user, per request.
4. **Stop spending on undifferentiated heavy lifting:** prefer managed services.
5. **Analyze and attribute expenditure:** tags and accounts per team.

The practical levers:

Lever	Action	Typical tool
Right-sizing	Shrink over-provisioned instances	Compute Optimizer, Cost Explorer recommendations
Pricing model	Commit for steady load, Spot for interruptible	Savings Plans, RIs, Spot
Elasticity	Scale in when idle; schedule off-hours shutdown	Auto Scaling, Instance Scheduler
Storage tiering	Move cold data to cheaper classes	S3 lifecycle, Intelligent-Tiering, gp2→gp3
Remove waste	Unattached EBS, idle load balancers, old snapshots, unused Elastic IPs	Trusted Advisor
Data transfer	Keep traffic in one AZ/Region, use CloudFront and VPC endpoints	Cost Explorer by usage type
Architecture	Serverless and managed services for spiky loads	Lambda, Fargate, Aurora Serverless
Processor	Graviton (ARM) instances, often ~20% better price-performance	EC2, RDS, Lambda

8.2 Cost management tools

- **Cost Explorer:** visualize, filter by service/account/tag, 12-month forecast, RI and Savings Plan recommendations.
- **AWS Budgets:** cost, usage, RI/SP utilization and coverage budgets; alerts by email/SNS; **Budget Actions** can apply an IAM/SCP policy or stop instances.
- **Cost Anomaly Detection:** ML finds unusual spend and alerts.
- **Cost and Usage Report (CUR / Data Exports):** the most detailed line-item data, delivered to S3; query with Athena or QuickSight.
- **Cost allocation tags** and **Cost Categories:** group spend by team, project, environment.
- **Trusted Advisor:** checks for cost, performance, security, fault tolerance, service limits.
- **Compute Optimizer:** right-sizing recommendations for EC2, ASG, EBS, Lambda, ECS.

8.3 The three pillars of observability

Pillar	Answers	AWS service
Metrics	Is it healthy? How much?	CloudWatch Metrics
Logs	What happened exactly?	CloudWatch Logs
Traces	Where is the slowness across services?	AWS X-Ray / CloudWatch Application Signals

Plus audit (who did what) with **CloudTrail** and configuration history with **AWS Config**.

8.4 Amazon CloudWatch in depth

- **Metrics:** namespaced, with dimensions. EC2 basic monitoring is every 5 minutes (detailed: 1 minute). Memory and disk usage are *not* default EC2 metrics — install the **CloudWatch agent**.
- **Custom metrics:** publish your own (e.g. orders per minute); high-resolution down to 1 second.
- **Alarms:** states OK / ALARM / INSUFFICIENT_DATA; actions to SNS, Auto Scaling, EC2 (stop, reboot, recover). **Composite alarms** combine alarms to reduce noise.
- **Logs:** log groups and streams, retention settings, **metric filters** (turn log patterns into metrics), **Logs Insights** (query language), subscription filters to Lambda/Firehose/OpenSearch.
- **Dashboards:** cross-account, cross-Region views.
- **Synthetics canaries:** scripted checks of endpoints and user flows. **RUM:** real user monitoring in browsers.
- **Container Insights / Lambda Insights:** detailed metrics for ECS, EKS, Lambda.

flowchart LR

```

APP[EC2 / Lambda / ECS] --> M[CloudWatch Metrics]
APP --> L[CloudWatch Logs]
L --> MF[Metric filter] --> M
M --> AL[Alarm]
AL --> SNS[SNS: email / Slack]
AL --> ASG[Auto Scaling]

```

Logs become metrics, metrics trigger alarms, and alarms either notify people or act automatically.

8.5 CloudTrail, Config and X-Ray

Service	Records	Example question
CloudTrail	API calls: who, what, when, from where	"Who deleted this S3 bucket?"
AWS Config	Resource configuration over time + compliance rules	"Was this security group open to 0.0.0.0/0 last Tuesday?"
X-Ray	Request traces across services	"Which downstream call makes checkout slow?"

CloudTrail details: event history keeps 90 days of management events for free; create a **trail** to store longer in S3 (enable log file validation, send to CloudWatch Logs). **Data events** (S3 object-level, Lambda invoke) are off by default and cost extra. **CloudTrail Lake** lets you query events with SQL. Use an **organization trail** to capture all accounts.

Config details: **Config rules** (managed or custom) check compliance; **remediation** with SSM Automation; **conformance packs** bundle rules; aggregators give a multi-account view.

8.6 Other monitoring services

- **AWS Health Dashboard**: AWS service events affecting your account.
- **Managed Service for Prometheus / Grafana**: open-source-compatible monitoring.
- **OpenTelemetry (ADOT)**: vendor-neutral instrumentation for traces and metrics.

8.7 Exam traps

- "Monitor memory usage on EC2" → CloudWatch agent (custom metric).
- "Who made this API call?" → CloudTrail. "What did the resource look like before?" → Config.
- "Alert when monthly cost will exceed \$500" → AWS Budgets with forecasted alert.
- "Find the slow microservice" → X-Ray.
- "Count ERROR lines in logs and alarm" → metric filter + alarm.
- "Cheapest for steady compute across EC2, Fargate and Lambda" → Compute Savings Plan.

8.8 Self-check

- ☐ Name 6 cost-reduction levers with an example each
- ☐ Compare CloudWatch, CloudTrail, Config, X-Ray
- ☐ Build a CPU alarm that notifies by email and scales out
- ☐ Explain management vs data events in CloudTrail
- ☐ Set up a Budget with forecasted alert (hands-on)

09 — Security

AWS security is layered defense: identity first, then network, data protection, detection and automated response — all under the shared responsibility model.

9.1 Shared responsibility model

AWS: security <i>of</i> the cloud	You: security <i>in</i> the cloud
Physical data centers, hardware, global network	IAM users, roles, policies, MFA
Hypervisor and host OS	Guest OS patching on EC2
Managed service infrastructure (RDS engine patching, Lambda runtime)	Security groups, NACLs, network design
	Data classification and encryption choices
	Application code and configuration

The more managed the service (EC2 → RDS → Lambda → S3), the more AWS takes on; you always own identity, data and configuration.

9.2 Well-Architected security principles

1. Strong identity foundation (least privilege, central identity, no long-term keys).
2. Traceability (log and monitor everything).
3. Security at all layers (edge, VPC, subnet, instance, app, data).
4. Automate security best practices (IaC, guardrails).
5. Protect data in transit and at rest.
6. Keep people away from data (automation over manual access).

7. Prepare for security events (runbooks, game days).

9.3 Data protection and encryption

AWS KMS manages encryption keys and integrates with almost every service; every use is logged in CloudTrail.

- Key types: **AWS owned** (invisible to you), **AWS managed** (aws/s3 , free, auto-rotated yearly), **customer managed** (you control policy, rotation, deletion).
- **Envelope encryption**: KMS key encrypts a data key; the data key encrypts the data. Keeps large data out of KMS (4 KB direct-encrypt limit).
- **Key policies** are the primary access control for a KMS key; cross-account use needs the key policy to allow it.
- Keys are Regional; **multi-Region keys** exist for cross-Region encryption.
- **CloudHSM**: dedicated single-tenant FIPS 140-2 Level 3 hardware when you must control the HSM.

Protect	Service
Data at rest	KMS encryption on S3, EBS, RDS, DynamoDB, EFS
Data in transit	TLS certificates from ACM (free public certs, auto-renew) on ALB, CloudFront, API Gateway
Secrets (DB passwords, API keys)	Secrets Manager (automatic rotation, e.g. RDS) or SSM Parameter Store SecureString (cheaper, no built-in rotation)
Sensitive data discovery	Macie (finds PII in S3)

Note: an unencrypted RDS instance cannot be encrypted in place — snapshot, copy the snapshot with encryption, restore.

9.4 Network and edge protection

- **AWS WAF**: layer-7 firewall on CloudFront, ALB, API Gateway, AppSync, Cognito. Blocks SQL injection, XSS, bad IPs, geo, rate-based rules; managed rule groups.
- **AWS Shield Standard**: free, automatic layer 3/4 DDoS protection.
- **Shield Advanced**: paid; enhanced DDoS protection, 24/7 Shield Response Team, cost protection for scaling during attacks.

- **AWS Network Firewall:** stateful VPC firewall, IDS/IPS, domain filtering.
- **Firewall Manager:** central management of WAF, Shield, security groups and Network Firewall across an Organization.
- Plus security groups, NACLs, private subnets, VPC endpoints (Topic 04).

9.5 Detection and response

Service	What it does
GuardDuty	ML threat detection from CloudTrail, VPC Flow Logs, DNS logs, EKS, S3, malware scanning; findings like crypto-mining or compromised keys
Inspector	Automated vulnerability scanning of EC2, ECR container images, Lambda
Macie	Discovers and classifies sensitive data in S3
Security Hub	Central dashboard; aggregates findings; runs CIS / AWS Foundational best-practice checks
Detective	Investigates root cause with graph analysis of findings
Config	Compliance of resource configurations (Topic 08)
CloudTrail	Audit trail of API calls (Topic 08)
Audit Manager / Artifact	Evidence for audits; AWS compliance reports (SOC, PCI, ISO)

flowchart LR

```

GD[GuardDuty finding] --> SH[Security Hub]
INS[Inspector] --> SH
MAC[Macie] --> SH
SH --> EB[EventBridge rule]
EB --> L[Lambda / SSM<br/>auto-remediate]
EB --> SNS[SNS alert]

```

Findings flow into Security Hub, and EventBridge triggers automatic fixes (e.g. isolate an instance, revoke a key) and alerts.

9.6 Incident response basics

1. **Prepare:** logging on everywhere, runbooks, break-glass access.
2. **Detect:** GuardDuty, Security Hub, alarms.
3. **Contain:** replace the security group with an isolation group, disable/rotate compromised keys, snapshot disks for forensics.
4. **Eradicate and recover:** rebuild from clean AMIs/IaC.
5. **Learn:** post-incident review, new guardrails.

If access keys leak: deactivate them immediately, check CloudTrail for what they did, rotate, and remove any resources the attacker created.

9.7 Exam traps

- "Protect website from SQL injection" → WAF. "DDoS with expert response team" → Shield Advanced.
- "Store DB password with automatic rotation" → Secrets Manager.
- "Detect compromised EC2 communicating with known bad IPs" → GuardDuty.
- "Scan EC2 and containers for CVEs" → Inspector. "Find credit card numbers in S3" → Macie.
- "Single place to view security posture across accounts" → Security Hub.
- "Who is responsible for patching the RDS database engine?" → AWS. "Patching the EC2 OS?" → You.

9.8 Self-check

- ☐ Explain shared responsibility for EC2 vs RDS vs Lambda
- ☐ Explain envelope encryption in your own words
- ☐ Compare GuardDuty, Inspector, Macie, Security Hub, Detective
- ☐ Compare WAF, Shield Standard, Shield Advanced, Network Firewall
- ☐ Write the steps for a leaked access key incident

10 — Certification

Topics 01–09 map directly to the AWS Certified Solutions Architect – Associate exam; the fastest path is Cloud Practitioner (optional) → Solutions Architect Associate, then a specialty or professional exam. Exam details below are approximate from general

knowledge — confirm codes, fees and domain weights in the official exam guide before booking.

10.1 Certification paths

Level	Certification	Who it's for
Foundational	Cloud Practitioner (CLF)	Beginners; business + basic tech
Foundational	AI Practitioner (AIF)	AI/ML concepts on AWS
Associate	Solutions Architect (SAA)	Designing systems — best first technical cert
Associate	Developer (DVA)	Building with Lambda, DynamoDB, APIs, CI/CD
Associate	SysOps / CloudOps Administrator	Operations, monitoring, automation
Associate	Data Engineer, ML Engineer	Data pipelines, ML workloads
Professional	Solutions Architect Pro, DevOps Engineer Pro	2+ years deep experience
Specialty	Security, Advanced Networking, and others	Deep single-domain skills

As a full-stack engineer, a good path is **SAA → Developer Associate → DevOps Pro or Security Specialty**.

10.2 Solutions Architect Associate: exam shape

- About 65 questions (some unscored), 130 minutes, multiple choice and multiple response.
- Scaled score 100–1000; passing ~720. No penalty for wrong answers — never leave blanks.
- Online proctored or at a test center (Pearson VUE); certification valid for 3 years.

Domain	Approx. weight	Topics in this guide
Design secure architectures	~30%	01, 02, 04, 09
Design resilient architectures	~26%	03, 04, 05, 06

Domain	Approx. weight	Topics in this guide
Design high-performing architectures	~24%	03, 05, 06
Design cost-optimized architectures	~20%	01, 05, 08

10.3 Keyword → answer map

Exam questions hide the answer in a few words. Learn these reflexes:

Keyword in question	Usually points to
"Least operational overhead"	Managed / serverless service
"Most cost-effective"	Cheapest option that still meets requirements (Spot, S3 IA, lifecycle, Savings Plans)
"Highly available"	Multi-AZ, ALB + ASG across AZs
"Disaster recovery, another Region"	Cross-Region replication, Aurora Global, backup copy
"Decouple"	SQS / SNS / EventBridge
"Real-time streaming"	Kinesis
"Temporary credentials"	IAM role / STS
"Shared file system"	EFS (Linux) / FSx (Windows)
"Private connectivity to S3"	Gateway VPC endpoint
"Audit API calls"	CloudTrail
"Protect against SQL injection"	WAF
"Millisecond reads, cache"	ElastiCache / DAX

10.4 Disaster recovery strategies (a favorite exam topic)

Strategy	RTO / RPO	Cost	Description
Backup and restore	Hours	Lowest	Backups copied to another Region; rebuild on disaster

Strategy	RTO / RPO	Cost	Description
Pilot light	Tens of minutes	Low	Core (database) always replicated; servers off until needed
Warm standby	Minutes	Medium	Scaled-down full copy running
Multi-site active-active	Near zero	Highest	Full capacity in 2+ Regions

RTO = how long to recover; RPO = how much data you can lose.

10.5 Question-solving method

1. Read the **last sentence** first — it states what is being asked.
2. Underline constraints: cost, performance, least effort, compliance, time.
3. Eliminate answers that are technically wrong (usually two).
4. Of the remaining two, pick the one that meets **every** requirement with the most managed approach.
5. Flag long questions, move on, return at the end. Aim for ~2 minutes per question.

10.6 Four-week study plan (after finishing Topics 01–09)

Week	Focus	Output
1	Re-read exam traps and self-checks of all 9 topics	List of weak areas
2	Hands-on labs on weak areas; build a 3-tier VPC app and a serverless API	Two working projects
3	Practice exams (timed) + review every wrong answer	Scores trending above 75%
4	Final practice exams, whitepapers skim, rest before exam day	Book exam when 2 practice tests $\geq$ 80%

Key resources: official exam guide and sample questions, AWS Skill Builder, AWS Well-Architected Framework whitepaper, AWS FAQs for S3, EC2, VPC, RDS, and a reputable practice-exam set.

10.7 Final checklist before booking

- ☐ Can explain every self-check item in Topics 01–09
- ☐ Can draw: multi-account org, 3-tier VPC, ALB+ASG, serverless app, DR options
- ☐ Scored $\geq 80\%$ on two different timed practice exams
- ☐ Know the four DR strategies and their RTO/RPO
- ☐ Know the keyword $\rightarrow$ answer map by heart
- ☐ ID ready; for online exams, room and webcam tested